Algorithm 1 ExpHash Split Operation

```
1: procedure INSERT_KV_ITEM
2:     Input: split_request    Output: FINISH/UNFINISH
3:     segment ← split_request.target
4:     depth ← split_request.depth
5:     bucket_migrate_count ← Get_Bucket_Number_For_Migration(split_request.granularity)    ▷ Segment-grained migrates all buckets, bucket-grained migrates several buckets
6:     src_chunk ← Get_Bottom_Chunk(segment)
7:     (dest_chunk, level) ← Get_Top_Chunk(segment)
8:     if is_Cross_Level_Migration_Triggered(split_request) then
9:         level ← level + 1
10:        dest_chunk ← Alloc_Attach_Higher_Level_Chunk(segment, level)
11:    second_phase_slots ← []
12:    for each bucket in src_chunk do
13:        if bucket is MIGRATION_DONE then
14:            continue                    ▷ ExpHash uses migration-done cursor to prevent such individual check
15:        dest_bucket_head ← Get_Associated_Buckets(bucket, dest_chunk, level)
16:        prefetch_next_bucket_set(bucket, dest_bucket_head, level)
17:        for each slot in bucket do
18:            Label MIGRATING_ITEM:
19:            if slot is END then
20:                break                                    ▷ All slots invalidated by concurrent splits
21:            else if slot is VACANT and Try_Mark_Slot_End(slot) then
22:                break                                    ▷ Invalidate all remaining slots
23:            else if slot is INVALID then
24:                continue
25:            else
26:                kv ← Get_KV_Item(slot)
27:                if is_Intact_Key_Need_For_Fingerprint_Reconstruct(slot) then
28:                    prefetch(kv)
29:                    Push slot into second_phase_slots
30:                else
31:                    dest_bucket ← Get_Dest_Bucket(dest_bucket_head, slot.fingerprint, level)
32:                    (dest_bucket_head, level) ← Migrate_Item_To_Destination(dest_bucket, slot, level)    ▷ if dest bucket has no space, new upper-level will be attached
33:                    if not Try_Mark_Slot_Invalid(slot) then
34:                        goto MIGRATING_ITEM                ▷ Retry migration if concurrently modified
35:        if is_Open_Addressing_For_Deferrable_Chunk_Allocation(bucket) then
36:            adjacent_bucket ← Get_Bucket_For_Open_Addressing(bucket)
37:            (dest_bucket_head, level) ← Handle_Migration_For_Adjacent_Bucket(adjacent_bucket, dest_bucket_head, level)
38:        if second_phase_slots is not empty then
39:            (dest_bucket_head, level) ← Handle_Second_Phase_Migration(second_phase_slots, dest_bucket_head, level)
40:        bucket_migrate_count ← bucket_migrate_count − 1
41:        if bucket_migrate_count = 0 and not is_Last_bucket(src_chunk, bucket) then
42:            return UNFINISH    ▷ bucket-grained splits may not complete all migrations in segment
43:    (segment1, segment2) ← Divide_Into_Two_Segment(segment)
44:    Update_Directory_Entry(segment1, segment2, depth + 1)    ▷ Includes directory expansion if required
45:    if Get_Segment_Level(segment1) > 1 then
46:        Submit_Split_Request_To_Scheduler(segment1, depth + 1)
47:    if Get_Segment_Level(segment2) > 1 then
48:        Submit_Split_Request_To_Scheduler(segment2, depth + 1)
49:    return FINISH
```